A Dynamic Load Balancing Strategy with Adaptive Threshold Based Approach

Taj Alam
School of Computer & Systems Sciences
Jawaharlal Nehru University
New Delhi, India
tajhashimi@gmail.com

Zahid Raza
School of Computer & Systems Sciences
Jawaharlal Nehru University
New Delhi, India
zahidraza@mail.jnu.ac.in

Abstract— To meet the objective of minimizing the job execution time, parallel computing has to deal with a lot of issues which crop up while working with parallel code. These issues can result in bottleneck and restrict the behavior of parallel program in attaining an aforesaid speedup as suggested by Amdahl Gene. The most problematic issue that crops up is the distribution of workload in both the categories of parallel system viz. homogeneous and heterogeneous systems. This situation demands an effective load balancing strategy to be in place in order to ensure a uniform distribution of load across the board. The scheduling 'm' jobs to 'n' resources with the objective to optimize the QoS parameters while balancing the load has been proven to be NP-hard problem. Therefore, a heuristic approach can be used to design an effective load balancing strategy. In this paper, a centralized dynamic load balancing strategy using adaptive thresholds has been proposed for a parallel system consisting of multiprocessors. The scheduler continuously monitors the load on the system and takes corrective measures as the load changes. The threshold values considered are adaptive in nature and are readjusted to suite the changing load on the system. Therefore, the scheduler always ensures a uniform distribution of the load on the processing elements with dynamic load environment.

Keywords- Parallel and distributed systems, load balancing, threshold, turnaround time, central scheduling.

I. INTRODUCTION

Parallelism in the computing systems can be viewed at two levels viz. hardware and software level. At hardware level it can be realized in the form of multiplicity of the processing elements and/or functional units whereas for the software level it can be seen as the multiple modules of the job demanding execution that can be run in parallel. The efficiency of a parallel system is governed by the degree of matching between the hardware and the underlying software parallelism. More is this match better is the efficiency [1-2, 8- 10].

The primary goal of the parallel systems is to minimize the job execution time and hence the turnaround time. This can be ensured by exploiting the inherent parallelism in the job by distributing the entire workload on the available computational resources, thus allowing various modules of the job to run simultaneously [11-12]. Scheduling is the method by which threads, processes or data flows are given access to system resources e.g. processor time, communications bandwidth [1, 3-5, 9]. To effectively utilize the resources, the job should be scheduled in such a way that no resources are underutilized and that the turnaround time is minimized. An effective load scheduling is very important for a system to load balance a system effectively or achieve the target quality of service while addressing issues e.g. synchronization, communication overhead, data locality and scalability. Scheduling of jobs should be done in such a way that each computing node has its proper share of work so that eventually the job turnaround time can be minimized. Load balancing can be treated as a subset of scheduling where such process is adopted. Load balancing is a methodology to distribute workload across multiple computers, network links, central processing units, disk drives, or other resources, to achieve optimal resource utilization, maximize throughput, minimize response time, and avoid overload [3]. Load balancing results in an allocation of the system recourses to individual jobs for certain time periods while optimizing the given objective function(s). In order to achieve the above goal load balancing strategy must exhibit the following features [3]:

(i) Must create little traffic overhead
(ii) Low overhead for running the load balancing algorithm
(iii) Must be fair so that heavily loaded node is balanced first with lightly loaded node
(iv) Load balancing should utilize minimum CPU time

The load balancing strategies can be broadly classified into centralized/ decentralized, static / dynamic, periodic / non periodic and with threshold / without threshold [13-19]. A load balancing policy can be either one or a combination of the above. Since, load on the system is bound to change with time; an adaptive load balancing policy is the best to work with as it addresses the problem of changing load. This can be ensured by defining thresholds for the workload on the system. The system reacts by redistributing the load as soon as these threshold values get crossed thereby ensuring a uniform distribution of the workload all the time. Further, an adaptive threshold based load balancing policy even ensures that the computational resources are utilized in the most optimum way.

II. RELATED WORK

The issue of load balancing has gained attention of many researchers. Therefore, a number of load balancing strategies using various approaches have been reported in the literature. A dynamic load balancing mechanism for

978-1-4673-2925-5/12/$31.00 ©2012 IEEE

distributed system is proposed in [13] with adaptive threshold where central node is used for maintaining load state information and decision for balancing is taken at local nodes. Six load balancing strategies are studied in [14] with application on four problems. These schemes include random, round robin, central load manager, threshold, central queue and local queue. In [15] various strategies for dynamic load balancing are explored which include sender initiated diffusion, receiver initiated diffusion, hierarchical balancing method, gradient model, domain exchange method. A simple load balancing strategy for task allocation in parallel machine has been proposed in [16] where load balancing is decentralized and execution of load balancing is decided among processors using the local queue length of individual processor. The processor with minimum queue length is given task of executing the load balancing. A comparison of three approaches of guided self scheduling, irregular parallel programs and lazy task creation without taking data locality into consideration has been done in [17] employing dynamic load balancing scheme implementing central queue and local queue while considering data locality problem.

III. SYSTEM PARAMETERS AND PROPOSED STRATEGY

The proposed model presents a centralized dynamic load balancing strategy which continuously keeps a track of the load on the nodes using threshold with the aim of minimizing the turnaround time of the jobs submitted for execution.

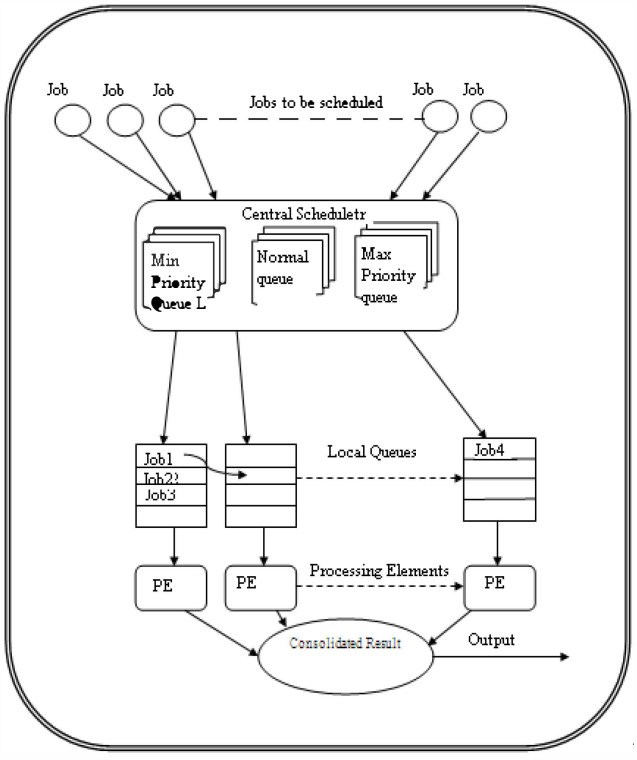


Fig. 1 – Scheduler Architecture

Since the model follows centralized job scheduling approach, of the available nodes, one node is taken as central scheduler on which job has got submitted and is eventually used for dispatching the independent job modules to the other nodes (processing elements). Each processing element has a local queue where the allotted jobs are queued up and are taken up for execution one by one in the order of their arrival. The scheduler under consideration has been shown in Fig. 1.

The model uses the centralized approach for load balancing. Thus, out of the nodes selected for job execution, one node is used as central scheduler which serves the two objectives viz. dispatching the jobs to the remaining nodes and making load balancing decisions depending on the system state. The remaining nodes simply act as processing elements for jobs execution. Each processing element has a local queue where jobs can be queued.

Various jobs can be submitted to the scheduler at the same time. Further, new jobs can be added up continuously while the older ones can keep finishing. Thus it is a completely dynamic job scenario with a changing load. Each job submitted for execution, in turn, can be considered to be comprising of sub-modules which can run in parallel. For a simple picture of the scheduler with one job submitted for execution, the process starts by randomly allocating these sub-modules of the job(s) to the processing elements. This random allocation results in a possible scenario in which few of the nodes gets a large number of sub-jobs to execute while some may get very less or no module to execute thus demanding load balancing which becomes the additional responsibility of the central scheduler. Whenever, the model experiences an uneven distribution of load, a readjustment of load is initiated to evenly distribute the load over the nodes till a balanced state is reached.

The load on the nodes is evaluated by using two threshold values viz. Lower Threshold (T_{lower}) and Upper Threshold (T_{upper}) values which are adaptive by nature. As a node is assigned some workload, the same enters its job execution queue. The global queues are maintained by the central scheduler only and are implemented as maximum priority queue for heavily loaded nodes and minimum priority queue for lightly loaded nodes. As the load on the system changes these thresholds are adjusted to suite the changing load on the system making the threshold selection adaptive i.e. the threshold values increases with increasing load and vice versa. As the average number of jobs in local queues of processing element increases, the threshold values are readjusted and so the global queues regarding the normal loaded nodes, lightly loaded nodes and heavily loaded nodes are adjusted. The load balancing process is instantaneous. As soon as the heavily loaded nodes and lightly loaded nodes are reported, the central scheduler starts load balancing.

The various parameters used in the model are presented in Table 1 along with their description.

Table 1-Parameter Used in the Model

Parameters	Description
K	Number of nodes
J	Number of jobs
J_i	Job identifier where 1 <= i <= J
N_i	Node identifier where 0 <= i <= K-1
l_i	Workload on each node N_i
T_{lower}	Lower threshold
T_{upper}	Upper threshold
LHM	Lower half mean of l_i for the nodes sorted in ascending order
UHM	Upper half mean of l_i for the nodes sorted in ascending order
M	Mean of l_i for the nodes sorted in ascending order
L	Min priority queue containing node identifier for nodes having load l_i below T_{lower}
H	Max priority queue containing node identifier for nodes having load l_i above T_{upper}
X	Queue for nodes having load between T_{lower} & T_{upper}
LQ_i	Local queue of jobs for each processing element N_i
LQL_i	Length of the local queue for each processing element N_i

Since the scheduler load balances the workload using thresholds, these values for under loaded nodes and overloaded nodes are considered as T_{lower} and T_{upper} which can be calculated using Lower Half Mean (LHM), Upper Half Mean (UHM), and Mean M values. To estimate LHM, UHM and M, initially the nodes are sorted in ascending order of their workloads with the condition $l_i >= l_{i-1}$ and then calculated using equations (i) – (iii) as

$$LHM = 1/((K-1)/2) \sum_{i=1}^{(K-1)/2} l_i \quad \text{----------(i)}$$

$$UHM = 1/((K-1)/2) \sum_{i=(K-1)/2+1}^{K-1} l_i \quad \text{----------(ii)}$$

$$M = 1/(K-1) \sum_{i=1}^{K-1} l_i \quad \text{----------(iii)}$$

Using equations (i) – (iii), T_{lower} and T_{upper} can be calculated as equations (iv)-(v).

$$T_{lower} = \begin{cases} LHM: LHM >= 0.9M \\ 0.9\,M: LHM < 0.9M \\ 1: LHM < 1,\ 0.9M < 1 \end{cases} \quad \text{---------- (iv)}$$

$$T_{upper} = \begin{cases} UHM: UHM <= 1.1M \\ 1.1M: UHM > 1.1M \\ 2: UHM < 2,\ 1.1\,M < 2 \end{cases} \quad \text{---------- (v)}$$

In proposed model both T_{upper} and T_{lower} are adaptive in nature. LHM and UHM provide us the reference points using which T_{lower} and T_{upper} are set. The scheduler works with the intention of bringing that state of the system in which both LHM and UHM (and hence T_{lower} and T_{upper}) ranges between ±10% of the mean M resulting in a load balanced state. If LHM and UHM are outside this range T_{lower} and T_{upper} are set to be 90% and 110% of M respectively. Thus the scheduler continues to load balance the system to bring the average workload between ±10 percent of the mean M.

Initially the values of thresholds T_{lower} and T_{upper} are taken as 1 and 2 respectively and are gradually adjusted using the node's workload sorted in the ascending order. Now, as the load on a node increases the value of thresholds are readjusted and accordingly the number of nodes in L, H and X keeps on changing. Nodes belonging to L, H and X can be decided using equation (vi), (vii) and (viii) respectively.

$N_i \in L$ *if* $l_i < T_{lower}$ ---------- *(vi)*

$N_i \in H$ *if* $l_i > T_{upper}$ ---------- *(vii)*

$N_i \in X$ *if* $l_i >= T_{lower}$ & $l_i <= T_{upper}$ ---------- *(viii)*

```
Load_Balancer ()

{   Set N0 as the Central Dispatcher
    Tlower =1
    Tupper = 2
    LQLi =0
    Move all nodes to minimum priority queue L
    For N0
      {
          Do
        {
           Dispatch          // randomly allocates jobs to nodes
           Update LQLi       // Update queue length with each
                             //   allocation
           Estimate  Tlower, Tupper
           Populate L and H  // using eq. (vi-vii)
         }
        If (H≠NULL and L≠NULL)
          {
            Transfer Jobs   // Transfer jobs from overloaded
                            // node to under loaded node
                            //using eq. (ix)
            Execute (Ji)    // Execute the jobs allocated
          }
        } while (LQLi≠0)
}
```

As the values of LHM and UHM approaches M the system approaches balanced state with even distribution of workload. The process of load redistribution continues for remaining number of nodes in L and H, reporting lightly loaded and heavily loaded status until either of the queue L or H becomes empty. Simultaneously, the threshold values are also adjusted with the changing queue lengths thereby changing the values in H, L and X as well. In this way using work offloading as basic load redistribution strategy, no node is idle if extra load is there on any node in a system. The number of jobs that are transferred from a heavily loaded node to the lightly loaded node is governed using equation (ix).

Number of jobs to be transferred= $(l_{i \in H} - l_{j \in L})/2$ --- (ix)

The algorithm for the same is presented in the box above.

IV. ILLUSTRATIVE EXAMPLE

To better understand the model, an example is illustrated in this section to present the basic working of the model in terms of the turnaround time computation. The example considers that no new job is added to local queue of a node and no job is taken away from the queue until the load balancing is done making it static whereas in practice, the model performs load balancing on the workload dynamically. In other words, the model considers the job service rate to be less than job arrival rate leading to removal of no job from the queue until the allotment has been done.

The example considers a scenario with a total number of available nodes for execution as 11. As per the scheduling strategy, N_0 acts as the central node and N_1 to N_{10} acting as the processing elements for job execution. Load on each node N_i is represented by l_i. Initially T_{lower} and T_{upper} are assumed to be 1 and 2 respectively. Total 134 jobs are assumed to be submitted to the system for execution and the allotment after random distribution of the load is as shown in the Table 2. Therefore, each entry in the table opposite to the node identifier indicates the number of jobs assigned to a node. The allotment clearly suggests an unbalanced state of the system thus prompting the scheduler to take corrective measures.

Table 2-Initial Allocation of Load

N_1	N_2	N_3	N_4	N_5	N_6	N_7	N_8	N_9	N_{10}
1	2	3	4	6	9	15	22	25	47

Initially the nodes are sorted in ascending order of their workload. In this case, the allotment is already in the sorted form. The process starts with the calculation of LHM, UHM and M. Using equation (i), LHM is calculated as the mean of workload on N_1, N_2, N_3, N_4 and N_5 which are the nodes in the lower half of the table sorted in ascending order and is calculated as

$$LHM = (1+2+3+4+6)/5 = 3.2.$$

Similarly, UHM is also calculated as per equation (ii) which is the mean of load on N_6, N_7, N_8, N_9, and N_{10} and is calculated as

$$UHM = (9+15+22+25+47)/5 = 23.6.$$

The value of M is then calculated as per equation (iii) and is mean of load on N_1, N_2 N_3, N_4, N_5, N_6, N_7, N_8, N_9, and N_{10} which is calculated as

$$M = (1+2+3+4+6+9+15+22+25+47)/10 = 13.4.$$

It can be seen from the example till now that the system started with the threshold values T_{lower} and T_{upper} as 1 and 2 respectively. Here, the mean M of the workload is 13.4, requiring the T_{lower} and T_{upper} values to be modified to move the bias towards M which acts the average workload of the system. Since, the difference between LHM (3.2) and UHM (23.06) from M (13.4) is very large, it indicates that there are many nodes which are under loaded and overloaded necessitating the load balancing to continue. Accordingly using equations (iv) – (v), the new value of T_{lower} and T_{upper} can be calculated as

$$T_{lower} = \max(\max(LHM, 0.9M), 1) = \max(\max(3.2, 12.06), 1) = 12.06.$$

$$T_{upper} = \max(\min(UHM, 1.1M), 2) = \max(\min(23.6, 14.74), 2) = 14.74.$$

The nodes that come under L and H as per equation (vi) – (vii) becomes

L (N_1, N_2 N_3, N_4, N_5, N_6)
H (N_{10}, N_9, N_8, N_7)

It can be seen that node N_1 is the most lightly loaded node with N_{10} being the most heavily loaded node. Thus node N_1 is workload balanced with N_{10} as per equation (ix) by transferring some jobs from N_{10} to N_1. Similarly, N_2 is balanced with N_9, N_3 is balanced with N_8 and N_4 is balanced with N_7. This results in emptying the queue H. Therefore, the scheduler stops the load balancing for the moment. The resultant load on each node after redistribution is shown in Table 3.

Table 3: Load on Nodes after Balancing

N_1	N_2	N_3	N_4	N_5	N_6	N_7	N_8	N_9	N_{10}
24	13	12	9	6	9	10	13	14	24

The resultant nodes after load balancing are again sorted to calculate the new values of thresholds T_{lower} and T_{upper}. The nodes with new resultant load in sorted order are shown in Table 4. In practice, this process is dynamic as the jobs are added and removed from the queue simultaneously. However, the example just illustrates the working of model till load is balanced without considering the addition and removal of new jobs for the sake of simplicity.

Table 4- Sorted Nodes According to Load of Table 3

N_5	N_4	N_6	N_7	N_3	N_2	N_8	N_9	N_1	N_{10}
6	9	9	10	12	13	13	14	24	24

In the way as illustrated till now, the new values of LHM, UHM and M now becomes 9.2, 17.6 and 13.4 respectively. It can be seen now that difference between LHM and UHM with M has reduced considerably indicating some load balancing which can be observed from Table 4 as

well where the distribution of workload is more uniform as compared to the initial state. Similarly, the values of T_{lower} and T_{upper} are calculated as 12.06 and 14.74 respectively. The nodes that come under L are N_5, N_4, N_6 and N_7 and H are N_{10} and N_1. The load is again balanced and the result is shown in Table 5 with Table 6 showing in the sorted order.

Table 5-Load Redistribution of Nodes of Table 4

N_5	N_4	N_6	N_7	N_3	N_2	N_8	N_9	N_1	N_{10}
15	16	9	10	12	13	13	14	17	15

Table 6-Nodes in Sorted Order of Table 5

N_6	N_7	N_3	N_2	N_8	N_9	N_5	N_{10}	N_4	N_1
9	10	12	13	13	14	15	15	16	17

Again, the values of LHM, UHM and M are calculated and found as 11.4, 15.4 and 13.4 respectively with the values of T_{lower} and T_{upper} being 12.06 and 14.74 respectively. Now, the nodes that come under L are N_6, N_7 and N_3. The same for H becomes N_1, N_4, N_{10} and N_5. Table 7 presents the load balanced system after this step with Table 8 presenting the nodes in a sorted order according to their workload

Table 7- Load Redistribution of Nodes of Table 6

N_6	N_7	N_3	N_2	N_8	N_9	N_5	N_{10}	N_4	N_1
13	13	13	13	13	14	15	14	13	13

Table 8-Nodes in Sorted Order of Table 7

N_1	N_2	N_3	N_4	N_6	N_7	N_8	N_9	N_{10}	N_5
13	13	13	13	13	13	13	14	14	15

The new values of LHM, UHM and M are now calculated as 13, 13.8 and 13.4 respectively which are approximately equal. This is the driving condition which depicts the even distribution of load. The values of T_{lower} and T_{upper} are 13 and 13.8 respectively. Therefore, no node is found to be under L whereas nodes that come under H are N_5, N_{10} and N_9. The load is readjusted only when L and H are both non empty. Since L has become empty, no load balancing is needed any further. The nodes will execute the jobs allocated to them till each node executes 13 jobs. Load status after execution of 13 jobs is shown in Table 9.

Table 9-Nodes after 13 Jobs Execution

N_1	N_2	N_3	N_4	N_6	N_7	N_8	N_9	N_{10}	N_5
0	0	0	0	0	0	0	1	1	2

The new values of LHM, UHM and M are 0, 0.8 and 0.4 respectively. So the values of T_{lower} and T_{upper} are 1 and 2 respectively. The nodes under L are N_1, N_2, N_3, N_4, N_6 and N_7 and there is no node under H. This state presents the other extreme in which L is non empty and H is empty again indicating the balanced state. Therefore, the nodes carry on execution till all local queues become empty. The final allocation of the workload to the nodes after complete load balancing is shown in Table 10 presenting the nodes with exact number of jobs allocated and hence executed. It can be seen that this value is near 13.4 which is the mean M of the workload.

Table 10-Nodes with Number of Jobs Allotted / Executed

Node No	N_1	N_2	N_3	N_4	N_5	N_6	N_7	N_8	N_9	N_{10}
Allotted	1	2	3	4	6	9	15	22	25	47
Executed	13	13	13	13	13	13	13	14	14	15

In the above example, the number of jobs considered for execution is 134 with each job executing in 0.264 seconds. Therefore, the time taken by 134 jobs to execute sequentially on one processing elements becomes 35.376 seconds. However, for parallel execution, the total turnaround time (TAT) can be calculated as

TAT= Max number of jobs executed on any node * *Execution Time of a Single Job ----------(x)*

In the given example, since the maximum number of jobs executed on any processing element is 15 as shown in Table 10, the TAT using equation (x) can be calculated as

TAT= 15*0.264=3.96 seconds

The speedup for such a system can be calculated as the ratio of the time taken T_{seq} by the job when executed sequentially on a node to the time taken for parallel execution T_{par}

$$\text{Speedup 'S'} = T_{seq} / T_{par} \quad \text{---------- (xi)}$$
$$= 35.376 / 3.96$$
$$= 8.93$$

As can be seen, the speedup obtained is 8.9 indicating approximately 900 % faster execution of the job. Similarly, the normalized speedup can be stated as

$$\text{Efficiency } \xi = \text{Speedup/Number of nodes ---------- (xii)}$$
$$= 8.93/10 = 0.89$$

Thus, the system is resulting in an efficiency of 89% which can be treated as fairly good.

V. CONCLUSION

It is always desired from a computing system that it should execute the job in the fastest possible way. The work presents a scheduling strategy that allocates the modules of the job(s) over the nodes in such a way that the desired objective of minimizing the turnaround time is met. The proposed model is based on centralized dynamic load

balancing strategy using thresholds. The threshold values used here are adaptive in nature i.e. as the load on the system increases, threshold values are readjusted to suite the changing load on the system. The model works in such a way that the thresholds tend to converge the load towards the mean of the workload. These values becomes approximately equal when the load becomes evenly distributed depicting the balanced state of the system. Moreover, the load redistribution process is fair as load is first readjusted between heavily loaded node and lightly loaded node through the use of max priority queue and min priority queue. The balancing process utilizes minimum CPU time as redistribution is only carried out when lightly loaded and heavily loaded nodes are reported. In the present work, it has been assumed that if average of the workload is distributed and hence executed by the processing elements, best results can be realized in terms of the turnaround time. Even better solution can be obtained if the model is made more realistic by considering other issues related to load balancing like communication cost and data locality.

REFERENCES

[1] Barney, Blaise, "Introduction to Parallel Computing", Lawrence Livermore National Laboratory.

[2] Amdahl, Gene, "Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities", AFIPS Joint Computer Conference, 1967, pp 1-4.

[3] Michael, Pinado, “Scheduling: Theory, Algorithms, and Systems”, http://www.amazon.com/Scheduling-Theory-Algorithms-Systems-2nd/dp/0130281387

[4] Baker, K.R., "Introduction to Sequencing and Scheduling”, John Wiley, 1974.

[5] Tanenbaum, A., “Operating Systems”, PHI, 2004.

[6] http://www.netlib.org/utk/lsi/pcwLSI/text/node248.html

[7] Brucker, P., “Scheduling Algorithms”, Fifth edition, Springer, Heidelberg, 2007.

[8] Tanenbaum, A., “Parallel and Distributed Systems”, PHI, 2002.

[9] Foster, Ian, “Designing and Building Parallel Programs", Addison-Wesley Longman Publishing Co., 1995.

[10] Steen, A.J. Van Der, Dongarra Jack, "Overview of Recent Supercomputers":http://www.phys.uu.nl/~steen/web03/contents.html.

[11] Chapman, Barbara, Jost, Gabriele, Pas, Ruud van der, Kuck, David J., "Using Open MP: Portable Shared Memory Parallel Programming", The MIT Press, 2008.

[12] Gropp, William, Lusk, Ewing, Thakur, Rajeev, “Using MPI-2”, MIT Press, 1999.

[13] Youran, Lan, "A Dynamic Load Balancing Mechanism for Distributed Systems” J. of Computer Science & Technology, May 1996, Vol. 11, No. 3, pp 1-13.

[14] Dubrovsky, Alexander, Friedman, Roy, Schuster, Assaf, “Load Balancing in a Distributed Shared Memory System”, IBM, 2005.

[15] Willebeek, Marc, Reeves, Anthony P.,” Strategies for Dynamic Load Balancing on Highly Parallel Computers”, IEEE Transaction on Parallel systems Software, 1999, pp 51-59.

[16] Rudolph, Larry et. al., “A simple load balancing scheme for task allocation in parallel machines”, Proceedings of the third annual ACM symposium on Parallel algorithms and architectures, p.237-245, July 21-24, 1991

[17] Keckler, W. Stephen, “The Importance of Locality in. Scheduling and Load Balancing for. Multiprocessors”, IEEE Transactions on Software Engineering, May 1986, pp 675-680.

[18] Sakae, Yoshiaki et. al., “Preliminary Evaluation of Dynamic Load Balancing Using Loop Re-partitioning on Omni/SCASH”, Third IEEE International Symposium on Cluster Computing and the Grid, May 12-May 15 2000.

[19] Drosinos, Nikolaos, Koziris, Nectarios,”Load Balancing Hybrid Programming Models for SMP Clusters and Fully Permutable Loops”, Supercomputing IEEE 2000 Conference, 04-10 Nov. 2000, pp 10-10.